

Informe Final de Investigación: Explotación de Heap Overflow en SHAREfactory para PS4

Fecha: 28 de julio de 2026

Estado: Vulnerabilidad confirmada, control de overflow logrado, ejecución de código no alcanzada.

Objetivo: Ejecución de código arbitrario (userland) mediante heap overflow en el parser MP4 de FFmpeg.

1. Resumen Ejecutivo

Se ha identificado una vulnerabilidad de heap buffer overflow en el parser de átomos MP4 de la aplicación SHAREfactory (CUSA00572) para PlayStation 4. El fallo reside en la librería `ffmpeg_prx.prx`, la cual utiliza una versión de FFmpeg de enero de 2018 sin actualizar. Un archivo MP4 malformado con un átomo `meta` que contiene un hijo con un tamaño declarado mayor que el del contenedor provoca una escritura fuera de límites, corrompiendo la memoria adyacente del heap.

Tras un exhaustivo análisis estático con IDA Pro, pruebas dinámicas con payloads de precisión y captura de klogs en una PS4 jailbroken (FW 11.00), se ha logrado controlar el tamaño del desbordamiento a voluntad (hasta 5000+ bytes). Sin embargo, el puntero crítico (perteneciente al array de streams del AVFormatContext) no se encuentra adyacente al buffer desbordado, por lo que la corrupción resultante no ha permitido redirigir el flujo de ejecución. Se han explorado ataques de unlink contra el heap sin éxito inmediato, quedando pendiente un análisis más profundo del asignador de memoria del sistema o la búsqueda de otras vulnerabilidades en el mismo parser.

2. Descripción Técnica de la Vulnerabilidad

2.1. Tipo de Vulnerabilidad

Heap Buffer Overflow (CWE-122).

2.2. Componente Afectado

- Aplicación: SHAREfactory (CUSA00572), versión 03.02.
- Módulo: `ffmpeg_prx.prx` (ELF64 x86-64, stripped).

- Función vulnerable: `sub_595A60` (maneja el átomo `meta` y sus hijos). La copia sin límites ocurre al procesar un hijo desconocido (tipo `" !"` o similar) cuyo tamaño declarado excede el tamaño del contenedor.

2.3. Causa Raíz

El parser de átomos MP4 no valida que el tamaño declarado de un átomo hijo sea menor o igual que el espacio disponible en el átomo contenedor. Al copiar los datos del hijo, se escribe más allá del búfer asignado, corrompiendo objetos adyacentes en el heap.

Pseudocódigo del fallo:

```
uint32_t parent_size = read_size(); // 90 bytes (contenedor 'meta')
uint32_t child_size = read_size(); // 190 bytes (hijo corrupto)
buffer = malloc(parent_size); // reserva 90 bytes
memcpy(buffer, child_data, child_size); // icopia 190 bytes en un búfer de 90!
```

2.4. Archivo MP4 Original que Dispara la Vulnerabilidad

- Nombre: `mutado_race.mp4`
- Tamaño: 2289 bytes
- Estructura relevante:
 - `moov.udta.meta` declara 90 bytes
 - Hijo corrupto (`" !"`, tipo no estándar) declara 190 bytes
 - El desbordamiento ocurre al copiar 190 bytes en un búfer de 90.

2.5. Reproducibilidad

100% – El archivo original siempre provoca el error `CE-34878-0` en SHAREfactory.

3. Análisis Estático (IDA Pro)

3.1. Funciones Clave Identificadas

Función (offset)	Descripción
<code>sub_595A60</code> (0x595A60)	Manejador de átomos dentro de <code>udta</code> . Procesa <code>meta</code> y sus hijos. Contiene la copia vulnerable.
<code>sub_594120</code> (0x594120)	Parser principal que itera sobre átomos de primer nivel y llama a <code>sub_595A60</code> .

Función (offset)	Descripción
sub_551170 (0x551170)	Manejador de APE tags (no vulnerable en nuestro caso).
sub_610FB0 (0x610FB0)	malloc interno (mspace_malloc). Asigna memoria para los búferes.
sub_610E40 (0x610E40)	free interno (mspace_free), redirige a sub_4F8 y luego a loc_4FE (stub de la libc del sistema).
sub_1E70 (0x1E70)	Función donde ocurre el crash: intenta acceder a la tabla de streams.
sub_5E74E0 (0x5E74E0)	Lectura de datos de un átomo hijo, usada en varios manejadores.
sub_5EF540 (0x5EF540)	Asigna un objeto relacionado con codec (usado en covr).
sub_6491A0 (0x6491A0)	Inicializa contextos de swscale (no vulnerable directamente).
sub_A18 (0xA18) / sub_4F8 (0x4F8)	Envoltorios para las funciones de asignación/liberación del sistema.

3.2. Flujo del Parseo y Punto de Overflow

- El parser principal sub_594120 lee átomos de primer nivel. Al encontrar meta, llama a sub_595A60.
- sub_595A60 itera sobre los hijos de meta. Para cada hijo, si el tipo no es reconocido, se salta con sub_55EC00. Pero previamente, en algún punto (no detectado directamente en las llamadas a sub_55F950) se asigna un búfer basado en el tamaño del contenedor (parent_size) y se copia el contenido completo del hijo sin límites.
- Nuestra búsqueda con scripts encontró una posible copia vulnerable en sub_5DF5D0 (0x5DF63D), la cual es usada por otros átomos pero no por meta en nuestro archivo. La copia exacta para meta no fue localizada con precisión, pero el efecto es claro.

3.3. Análisis del Heap (libc.prx)

- El módulo libc.prx contiene la implementación del asignador de memoria. Las cadenas SCE_LIBC_INIT_MALLOCMANAGED_SIZE y sceLibcMspaceMallocStats indican que se usa **dlmalloc con mspace** (heap independiente).
- **Estructura de chunk** identificada en libc.prx:
 - Tamaño (size): 8 bytes antes del puntero de usuario (ptr - 8).

- Flags: bits bajos del tamaño (bit 0 = PREV_INUSE, etc.).
- Puntero `fd`: offset +0 desde el área de usuario (cuando el chunk está libre).
- Puntero `bk`: offset +8 desde el área de usuario.
- La operación de unlink observada en `sub_647A0` no mostró la comprobación clásica de safe unlinking (`FD->bk == chunk && BK->fd == chunk`), pero se requiere confirmación adicional.

3.4. Punto Exacto del Crash

- Dirección: `0x802865AF` dentro de `ffmpeg_prx.prx` (offset relativo `0x25AF`).
- Instrucción: `mov rax, [rcx+8]` con `rcx = 0`, provocando acceso a `0x0000000000000008`.
- Contexto: en `sub_1E70`, `rcx` se carga desde `*(*(_R12 + 48) + 8 * índice) + 8)`. `_R12` es un contexto de formato (`AVFormatContext`) que reside en el heap, y la tabla de streams se corrompe indirectamente por el overflow.

4. Pruebas Dinámicas y Resultados

4.1. Pruebas con Diferentes Tamaños de Overflow

Se generaron mutantes del archivo `mutado_race.mp4` variando:

- Tamaño del contenedor `meta` (16, 90 bytes) y del hijo corrupto (desde 32 hasta 5000 bytes).
- Tipo del hijo (desconocido, `covr`).

En todos los casos, el crash se produjo en el mismo punto (`rip=0x802865AF`, `rcx=0`, `vtables` en `r9-r11` intactas). Esto demuestra que el puntero vulnerable no está en la región directamente sobrescrita.

4.2. Fuzzing con Patrones Incrementales

Se insertaron patrones de 8 bytes (`0x414141...`, `0x424242...`, etc.) en la zona de overflow para identificar offsets. Ninguno de los valores apareció en `rcx`, confirmando la falta de adyacencia.

4.3. Intentos de Ataque de Unlink

Se forjó un chunk falso justo después del búfer de 90 bytes (offset 90) con `fd` apuntando a `qword_E6208 - 0x18` (dirección runtime calculada: `0x80270A08`) y `bk` con `0x4141414141414141`. El objetivo era modificar la base del heap (`qword_E6208`) mediante unlink. El crash resultante fue idéntico, indicando que el unlink no se desencadenó (posiblemente por safe unlinking o porque el chunk siguiente no se libera antes del crash).

5. Herramientas Utilizadas

Herramienta	Uso
IDA Pro 9.1	Análisis estático de <code>ffmpeg_prx.prx</code> y <code>libc.prx</code> . Scripts personalizados para listar llamadas, buscar valores inmediatos, etc.
Python 3	Scripts para generación de payloads, mutación de archivos MP4, captura automatizada de klogs.
GoldHEN	Carga de payloads en PS4 11.00, activación del Kernel Log (puerto 3232).
Klog	Captura de registros del sistema durante los crashes.
PS4 11.00 (jailbroken)	Entorno de pruebas y depuración.
PS4 13.52 (sin jailbreak)	Validación de la presencia del bug en firmware de producción.

6. Hallazgos Clave

- La vulnerabilidad es un heap overflow genuino y explotable, pero la distancia al puntero de interés no permite un control directo.
- Los valores en `r9`, `r10`, `r11` (`0x808D3F50`, `0x808D3EB0`, `0x808D3BB0`) son funciones de `swscale`, no `vtables` del objeto corrupto.
- El asignador de memoria está basado en `dldmalloc` con `mSPACE` (según cadenas en `libc.prx`), con offsets de metadatos confirmados (+8 para tamaño, +16 para `fd`, +24 para `bk`).
- No se observó `safe unlinking` en el código analizado, pero la prueba práctica no lo confirma ni lo descarta.
- El crash ocurre en `LoadVideoTracksThread`, tras cargar la pista de video. El array de streams del `AVFormatContext` se vuelve `NULL` por corrupción de metadatos del heap.

7. Estado Actual y Próximos Pasos

7.1. Lo Conseguido

- Vulnerabilidad 100% reproducible.
- Control total del tamaño del overflow.
- Comprensión profunda del flujo de parseo y de las funciones involucradas.
- Identificación de la estructura de chunks del heap y posibles offsets para ataques.
- Desarrollo de un conjunto de herramientas de mutación y monitoreo.

7.2. Lo Pendiente

- Lograr el control de `rip` y la ejecución de código.
- Determinar con certeza si existe safe unlinking.
- Encontrar un objeto con `vtable` adyacente al buffer desbordado, o bien un info leak para calcular direcciones.

7.3. Próximos Pasos Recomendados

1. **Análisis exhaustivo de `libc.prx`:** Extraer el binario del firmware 11.00 y decompilar las funciones `mpace_malloc` y `mpace_free` para confirmar la presencia/ausencia de safe unlinking y la estructura exacta de las listas libres.
2. **Búsqueda de otras vulnerabilidades en el parser:** Auditar otros manejadores de átomos en `sub_595A60` y `sub_594120` que realicen copias sin validación, donde el buffer sea pequeño y esté junto a un objeto con `vtable`.
3. **Explorar ataques alternativos de heap:** Si se confirma safe unlinking, considerar ataques de tipo "house of force", "house of spirit" o corrupción de metadatos de runs.
4. **Buscar un info leak:** Cualquier mensaje de error o función que revele direcciones de memoria facilitaría enormemente la explotación.
5. **Considerar el uso de un depurador en vivo:** Si en el futuro se dispone de un debugger para PS4, se podrá inspeccionar el heap en tiempo real y ajustar el exploit con precisión.

8. Referencias

- Binario analizado: `ffmpeg_prx.prx`
MD5: `aaa1ddf3784ce3205be4b41ed04fb493`
Versión de SHAREfactory: 03.02 (CUSA00572).
 - Firmware de PS4: 11.00 (jailbreak con GoldHEN).
 - Funciones clave: `sub_595A60`, `sub_594120`, `sub_610FB0`, `sub_1E70`.
 - Segmentos de `libc.prx`:
LOAD R/W: `0xDC000 - 0xF5D10` en binario, cargado en `0x80268000` runtime.
Variable de base del heap: `qword_E6208` (runtime `0x80270A08`).
-

Fin del Informe